

Quiz MCQ – Final Assessment

Course: Full Stack AI B 5 SAT,SUN)

Dated:3rd March 2026

Name: Fatima Ali

NIC No:35202-9129379-0

NumPy — 5 MCQs

1) What is the primary advantage of using NumPy arrays over Python lists for numeric computation?

- A. NumPy arrays are immutable
- B. NumPy arrays automatically parallelize operations
- C. NumPy arrays are contiguous and support vectorized operations
- D. NumPy arrays support nested dictionaries

Answer: C (NumPy arrays are contiguous and support vectorized operations)

2) What is the result shape of `np.sum(a, axis=0)` where `a` is shape `(3, 4)`?

- A. `(3, 4)`
- B. `(3,)`
- C. `(4,)`
- D. `(1, 4)`

Answer: C`(4,)`

3) Which statement about broadcasting is TRUE?

- A. Only arrays of the same shape can be added
- B. Trailing dimensions are compared and dimensions of size 1 are “stretched”
- C. Broadcasting only works with integer arrays
- D. Broadcasting always creates a copy

Answer: B(Trailing dimensions are compared and dimensions of size 1 are “stretched”)

4) Which creates a *view* most often (not guaranteed copy)?

- A. `a.flatten()`
- B. `a.copy()`
- C. `a.ravel()`
- D. `np.array(a)`

Answer:C`(a.ravel() )`

5) What does `-1` mean in `a.reshape(-1, 2)`?

- A. Use dynamic type inference
- B. Delete a dimension

- C. Infer that dimension from the total size
- D. Fill missing values with -1

Answer: C(Infer that dimension from the total size)

Python – List — 5 MCQs

1) What's the difference between *append* and *extend*?

- A. Both add multiple elements
- B. *append* adds an element; *extend* concatenates an iterable
- C. Both add a single element
- D. *extend* inserts at the beginning

Answer: B(*append* adds an element; *extend* concatenates an iterable)

2) What happens with `lst = [[0]] * 3; lst[0].append(1)`?

- A. Only `lst[0]` changes
- B. All inner lists change
- C. Raises an error
- D. Creates a deep copy

Answer: B(All inner lists change)

3) What's the result of `sorted([3, 1, 2])` vs `[3,1,2].sort()`?

- A. Both return a new list
- B. Both modify in place
- C. *sorted* returns new; *sort* modifies in place and returns *None*
- D. *sorted* modifies in place; *sort* returns a new list

Answer: C(*sorted* returns new; *sort* modifies in place and returns *None*)

4) What does `lst[::-1]` do?

- A. Slices every other element
- B. Reverse in place
- C. Returns a reversed slice
- D. Raises a `ValueError`

Answer: C(Returns a reversed slice)

5) Which removes by value (first occurrence)?

- A. `pop()`
- B. `del`
- C. `remove()`
- D. `clear()`

Answer: C(`remove()`)

Python – Tuples — 5 MCQs

1) Which creates a single-element tuple?

- A. (1)
- B. (1,)
- C. tuple(1)
- D. {1,}

Answer:

2) What is TRUE about tuples?

- A. Tuples are always hashable
- B. Tuples are immutable but can contain mutable elements
- C. Tuples are sorted collections
- D. Tuples cannot be nested

Answer:

3) What does `a, b = b, a` do?

- A. Swaps values without temp variable
- B. Raises a ValueError
- C. Deep copies both
- D. Creates a list

Answer:

4) Which is valid tuple unpacking with a starred expression?

- A. `a, *b, c = [1, 2, 3, 4]`
- B. `*a, *b = [1, 2, 3, 4]`
- C. `*a = [1, 2, 3, 4]`
- D. `a, b, * = [1, 2, 3, 4]`

Answer:

5) What is `type((1))`?

- A. `tuple`
- B. `list`
- C. `int`
- D. `set`

Answer:

Python – Set — 5 MCQs

1) How to create an empty set?

- A. `{}`
- B. `set()`
- C. `[]`
- D. `() => set`

Answer:

2) Which operation removes an element without raising if missing?

- A. `remove`
- B. `discard`
- C. `pop`
- D. `delete`

Answer:

3) What is $A \wedge B$ for sets A and B?

- A. Intersection
- B. Union
- C. Difference
- D. Symmetric difference

Answer:

4) Which is TRUE about sets?

- A. Sets are ordered
- B. Sets allow duplicates
- C. Membership check is average $O(1)$
- D. Sets can contain lists

Answer:

5) What is the difference between `set` and `frozenset`?

- A. `frozenset` preserves order
- B. `frozenset` is immutable and hashable
- C. `frozenset` allows duplicates
- D. `set` is immutable

Answer:

Python – Dictionary — 5 MCQs

1) What does `in` check for a dict `d`?

- A. Values
- B. Keys
- C. Items

D. Types

Answer:

2) How to safely access a missing key with default?

- A. `d['k']` or `default`
- B. `d.get('k', default)`
- C. `d.setdefault('k')`
- D. `d.pop('k', default)`

Answer:

3) Dicts preserve insertion order since:

- A. Python 3.5
- B. Python 3.6 (CPython impl detail) / guaranteed 3.7+
- C. Python 3.4
- D. Never

Answer:

4) Merge two dicts *a* and *b* (Python 3.9+)?

- A. `a + b`
- B. `a.update(b, merge=True)`
- C. `a | b`
- D. `merge(a,b)`

Answer:

5) Which can be a dict key?

- A. `list`
- B. `set`
- C. `dict`
- D. `tuple` of hashables

Answer:

pandas — 5 MCQs

1) Difference between `.loc` and `.iloc`?

- A. Both are label-based
- B. `.loc` is label-based; `.iloc` is integer-position based
- C. Both are position-based
- D. `.loc` is for columns only

Answer:

2) Default join type in `pd.merge(left, right, on='key')`?

- A. Left
- B. Right
- C. Inner
- D. Outer

Answer:

3) Which converts a long format to wide format?

- A. `melt`
- B. `pivot / pivot_table`
- C. `stack` only
- D. `explode`

Answer:

4) Which is best to apply different transforms to numeric vs categorical columns before modeling (sklearn)?

- A. `df.apply` with `axis=1`
- B. `pipeline`
- C. `sklearn.compose.ColumnTransformer`
- D. `df.transform` only

Answer:

5) Which reduces memory for repeated string-like categories?

- A. `astype('int8')`
- B. `astype('category')`
- C. `astype('object')`
- D. `astype('string')`

Answer:

Seaborn — 5 MCQs

1) Which Seaborn call sets theme globally (modern API)?

- A. `sns.set()`
- B. `sns.set_theme()`
- C. `sns.style.use()`
- D. `plt.style.use()`

Answer:

2) Which is a *figure-level* function (returns `FacetGrid`)?

- A. `sns.scatterplot`
- B. `sns.boxplot`

- C. *sns.relplot*
- D. *sns.violinplot*

Answer:

Explanation: Figure-level: *relplot*, *catplot*, *displot*, *lmplot*.

3) Which parameter creates color categories?

- A. *style*
- B. *hue*
- C. *col*
- D. *row*

Answer:

4) Which statement about *sns.distplot* is TRUE?

- A. It is the current recommended API
- B. It's deprecated; use *displot*/*histplot*/*kdeplot*
- C. It plots only kernel density
- D. It plots only histograms

Answer:

5) What does *sns.load_dataset("tips")* do?

- A. Loads a local dataset from pandas
- B. Downloads example dataset from online repository
- C. Creates a synthetic dataset
- D. Loads scikit-learn dataset

Answer:

scikit-learn — 10 MCQs

1) Which methods characterize the estimator interface?

- A. *fit*, *predict*, *transform* (where applicable)
- B. *run*, *score*, *export*
- C. *learn*, *forecast*
- D. *compile*, *fit*, *evaluate*

Answer:

2) Why use *Pipeline*?

- A. To parallelize training
- B. To chain preprocessing + model with correct CV behavior
- C. To store datasets
- D. To visualize models

Answer:

3) Proper scaling practice when using *StandardScaler*?

- A. Fit on full data, transform train/test
- B. Fit on test only
- C. Fit only on train; transform train and test
- D. No scaling is needed

Answer:

4) Best split for imbalanced classification?

- A. *train_test_split(..., shuffle=False)*
- B. *train_test_split(..., stratify=y)*
- C. Random split without seed
- D. KFold without shuffling

Answer:

5) What does *GridSearchCV* do?

- A. Randomly samples hyperparameters
- B. Exhaustively searches specified grid with CV
- C. Prunes features
- D. Performs PCA

Answer:

6) Which metric is typically NOT for regression?

- A. MAE
- B. MSE
- C. R^2
- D. F1-score

Answer:

7) Handling categorical variables in pipelines safely?

- A. *OneHotEncoder(handle_unknown='ignore')*
- B. *LabelEncoder* on entire DataFrame
- C. *DictVectorizer* for numeric-only columns
- D. Casting to *category* is sufficient

Answer:

8) Apply different preprocessing per column types?

- A. *FeatureUnion*
- B. *ColumnTransformer*
- C. *PolynomialFeatures*

D. *SelectKBest*

Answer:

9) What does *random_state* control?

- A. Model convergence speed
- B. Reproducibility of stochastic operations
- C. GPU usage
- D. Feature scaling range

Answer:

10) For SVMs and KNNs, feature scaling is:

- A. Unnecessary
- B. Harmful
- C. Recommended/Important
- D. Only for text data

Answer:

TensorFlow / Keras — 10 MCQs

1) *Sequential* vs Functional API?

- A. Sequential supports multiple inputs
- B. Functional supports arbitrary DAGs and multi-input/output
- C. Both only support linear stacks
- D. Functional is deprecated

Answer:

2) Proper setup for binary classification output layer & loss?

- A. *Dense(2, softmax), categorical_crossentropy*
- B. *Dense(1, sigmoid), binary_crossentropy*
- C. *Dense(1, relu), mse*
- D. *Dense(2, sigmoid), binary_crossentropy*

Answer:

3) *sparse_categorical_crossentropy* is appropriate when:

- A. Labels are one-hot vectors
- B. Labels are probability distributions
- C. Labels are integer class indices
- D. Regression target is sparse

Answer:

4) Which returns a history of losses/metrics per epoch?

- A. `model.evaluate`
- B. `model.fit`
- C. `model.predict`
- D. `model.compile`

Answer:

5) What does `EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)` do?

- A. Stops immediately after 3 epochs
- B. Stops if no improvement for 3 epochs and restores best weights
- C. Reduces learning rate
- D. Saves checkpoints

Answer:

6) For input pipelines, which improves performance?

- A. `dataset.prefetch(tf.data.AUTOTUNE)`
- B. `dataset.batch(1)` only
- C. Loading all data into RAM
- D. Using Python lists

Answer:

7) Which layer helps internal covariate shift and can stabilize training?

- A. `Dropout`
- B. `BatchNormalization`
- C. `Flatten`
- D. `Embedding`

Answer:

8) How to save a full Keras model (architecture+weights+optimizer state) in a portable way?

- A. `model.save('path')` (SavedModel format)
- B. `np.save(model)`
- C. `pickle.dump(model)`
- D. `model.to_json()`

Answer:

9) Custom training loop uses:

- A. `tf.function` only
- B. `tf.GradientTape()`
- C. `model.fit_loop()`

D. *tf.Run()*

Answer:

10) For sequence models with padded inputs, which helps ignore padding?

- A. *Masking* layer or *mask_zero=True* in *Embedding*
- B. *Dropout*
- C. *BatchNormalization*
- D. *ActivityRegularization*

Answer:

↔ LangChain / LLM — 5 MCQs

1) What does *PromptTemplate* provide?

- A. Tokenization
- B. Structured prompts with variables/placeholders
- C. GPU acceleration
- D. Model compression

Answer:

2) What is an *LLMChain*?

- A. A vector database
- B. A chain that combines a prompt template with an LLM to produce outputs
- C. A reinforcement learning agent
- D. An embeddings model

Answer:

3) Purpose of *Memory* (e.g., *ConversationBufferMemory*) in LangChain?

- A. Store API keys
- B. Maintain conversational context across turns
- C. Manage GPU memory
- D. Compress documents

Answer:

4) What does *RetrievalQA* typically combine?

- A. LLM + web browser
- B. LLM + Retriever (vector store) for context-augmented answers
- C. LLM + RL environment
- D. LLM + image generator

Answer:

5) Which component indexes and searches chunks via embeddings most directly?

- A. *Tool*
- B. *Agent*
- C. *VectorStore* (e.g., FAISS/Chroma)
- D. *Callback*

Answer: